

Example usage of Mega2R

Robert V. Baron and Daniel E. Weeks

June 16, 2017, 14:55

Contents

1	Load Libraries	1
2	Tutorial directory and files	1
3	Installation	2
4	Example Data	4
5	Using Mega2R packages	4
5.1	Creating a Mega2 database	4
5.2	Reading a Mega2 database into R	8
5.3	Running pedgene using the mega2pedgene R package	11
5.4	Writing a VCF File using the mega2vcf R package	13
5.5	Creating a GenABEL gwaa.data-class object using the mega2genabel R package	13
6	Regression	14
6.1	VCF regression	15
6.2	GenABEL regression	17
7	Next steps	23

1 Load Libraries

2 Tutorial directory and files

The files you will need for this tutorial are in the mega2rtutorial directory. You should `cd` there to do these exercises.:

You should see the following files:

..... | |

-----|-----|-----

[1] "MEGA2.BATCH.seqsimr" | "MEGA2.BATCH.srdta" | "MEGA2.BATCH.vcf"

[4] "Mega2r.map" | "Mega2r.ped" | "mega2r.Rmd"

[7] "mega2r.pdf" | "seqsimr.db" |

```
# Print the working directory
getwd()
```

```
## [1] "/Users/rbaron/mega2/bb/srcdir/R/mega2rtutorial"
```

```
# Print the files
list.files(pattern = "mega2.*|seqsimr.db", ignore.case = TRUE)
```

```
## [1] "MEGA2.BATCH.seqsimr" "MEGA2.BATCH.srdta" "MEGA2.BATCH.vcf"
## [4] "Mega2r.map"          "Mega2r.ped"         "mega2r.Rmd"
## [7] "mega2r.pdf"          "seqsimr.db"         "seqsimr.db.old"
```

3 Installation

To run these exercises, you should install the CRAN packages *mega2r*, *mega2vcf*, *mega2genabel* and *mega2pedgene*. Once they are in CRAN, the preferred way to install will be to use the `install.packages()` command. For now, we prefer to use `devtools` in R; as in

```
devtools::install("../mega2r")
```

```
## Installing mega2r
```

```
## '/Library/Frameworks/R.framework/Resources/bin/R' --no-site-file \
## --no-environ --no-save --no-restore --quiet CMD INSTALL \
## '/Users/rbaron/mega2/bb/srcdir/R/mega2r' \
## --library='/Library/Frameworks/R.framework/Versions/3.3/Resources/library' \
## --install-tests
```

```
##
```

```
## Reloading installed mega2r
```

```
## unloadNamespace("mega2r") not successful, probably because another loaded package depends on it.Forc
```

```
devtools::install("../mega2pedgene")
```

```
## Installing mega2pedgene
```

```
## '/Library/Frameworks/R.framework/Resources/bin/R' --no-site-file \
## --no-environ --no-save --no-restore --quiet CMD INSTALL \
## '/Users/rbaron/mega2/bb/srcdir/R/mega2pedgene' \
## --library='/Library/Frameworks/R.framework/Versions/3.3/Resources/library' \
## --install-tests
```

```
##
```

```
## Reloading installed mega2pedgene
```

```
devtools::install("../mega2vcf")
```

```
## Installing mega2vcf
```

```
## '/Library/Frameworks/R.framework/Resources/bin/R' --no-site-file \  
## --no-environ --no-save --no-restore --quiet CMD INSTALL \  
## '/Users/rbaron/mega2/bb/srcdir/R/mega2vcf' \  
## --library='/Library/Frameworks/R.framework/Versions/3.3/Resources/library' \  
## --install-tests
```

```
##
```

```
## Reloading installed mega2vcf
```

```
devtools::install("../mega2genabel")
```

```
## Installing mega2genabel
```

```
## '/Library/Frameworks/R.framework/Resources/bin/R' --no-site-file \  
## --no-environ --no-save --no-restore --quiet CMD INSTALL \  
## '/Users/rbaron/mega2/bb/srcdir/R/mega2genabel' \  
## --library='/Library/Frameworks/R.framework/Versions/3.3/Resources/library' \  
## --install-tests
```

```
##
```

```
## Reloading installed mega2genabel
```

Certainly,

```
R CMD install mega2r  
R CMD install mega2pedgene  
R CMD install mega2vcf  
R CMD install mega2genabel
```

will work, too. It may generate warnings about non-package related items in the tree. All these packages may fetch other dependent CRAN packages. We also require Bioconductor Annotations that must be loaded manually. The first line loads the Bioconductor loader and the next two lines load two annotation databases. One for gene transcript locations and one to map gene names to the entrez gene IDs. (Note: You may choose a different transcript database from Bioconductor or construct one of your own.) Please type in R:

```
source("https://bioconductor.org/biocLite.R")
```

```
## Bioconductor version 3.4 (BiocInstaller 1.24.0), ?biocLite for help
```

```
## A new version of Bioconductor is available after installing the most  
## recent version of R; see http://bioconductor.org/install
```

```

biocLite("TxDb.Hsapiens.UCSC.hg19.knownGene")

## BioC_mirror: https://bioconductor.org

## Using Bioconductor 3.4 (BiocInstaller 1.24.0), R 3.3.3 (2017-03-06).

## Installing package(s) 'TxDb.Hsapiens.UCSC.hg19.knownGene'

## installing the source package 'TxDb.Hsapiens.UCSC.hg19.knownGene'

## Old packages: 'backports', 'devtools', 'gdata', 'jsonlite', 'R6',
##   'RcppEigen', 'rmarkdown', 'survey', 'tibble', 'XML'

biocLite("org.Hs.eg.db")

## BioC_mirror: https://bioconductor.org

## Using Bioconductor 3.4 (BiocInstaller 1.24.0), R 3.3.3 (2017-03-06).

## Installing package(s) 'org.Hs.eg.db'

## installing the source package 'org.Hs.eg.db'

## Old packages: 'backports', 'devtools', 'gdata', 'jsonlite', 'R6',
##   'RcppEigen', 'rmarkdown', 'survey', 'tibble', 'XML'

```

4 Example Data

We have chosen to use the SeqSIMLA2 program to generate a data set for us. We needed to sub-sample the data down to 1,380 people and 1,000 markers to make the size manageable. These data will be used to illustrate Mega2 and Mega2R operations that follow.

Note: This simulator only produces markers on chromosome 1.

5 Using Mega2R packages

5.1 Creating a Mega2 database

We have provided files in the mega2rtutorial directory, that contain the data from the simulation. These files are in PLINK ped format data for 1,380 samples with 1,000 markers:

- seqsimr.ped
- seqsimr.map

You will need to obtain the Mega2 program <https://watson.hgen.pitt.edu/register/>. First, you will invoke Mega2 on your data. To make matters simple, we will use a pre-constructed Mega2 batch file to automate the processing by Mega2. To run Mega2 to process and create the SQLite3 database 'seqsimr.db', we issue this command at the Unix prompt:

```
## =====
##                               MEGA2 4.9.2
##   Copyright (C) 1999-2017 Robert Baron, Justin R. Stickel, Charles P. Kollar,
##   Nandita Mukhopadhyay, Lee Almasy, Mark Schroeder, William P. Mulvihill,
##   Daniel E. Weeks, and University of Pittsburgh
##
##   Last updated: Jun 13 2017, 09:36:42 , valid until June 15, 2018.
##   Compiled with gcc version 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)
##
##   Mega2 comes with ABSOLUTELY NO WARRANTY.
##   See LICENSE.txt for terms of copying, modifying & redistributing Mega2.
## =====
## NOTE: For humans, chromosome 23 codes for X, 24 codes for Y and 25 codes for XY.
##
## Run date:                    2017-6-16-14-55
##
## Running Mega2 in batch mode from MEGA2.BATCH.seqsimr.
## Input filenames and missing value indicator read in from batch file.
## Dump Analysis option read in from batch file.
## WARNING: Locus selections not specified in batch file.
## WARNING: Going to Reorder menu.
## WARNING: Trait selections not specified in batch file.
## WARNING: Going to Trait selection menu.
## =====
## Keyword Input_Locus_File not in batch file, Locus file assumed to be unspecified.
## Keyword Input_Map_File not in batch file, Map file assumed to be unspecified.
## Keyword Input_Omit_File not in batch file, Omit file assumed to be unspecified.
## Keyword Input_Frequency_File not in batch file, Frequency file assumed to be unspecified.
## Keyword Input_Penetrance_File not in batch file, Penetrance file assumed to be unspecified.
## Keyword Input_Aux_File not in batch file, Aux file assumed to be unspecified.
## Keyword Input_Phenotype_File not in batch file, Phenotype file assumed to be unspecified.
## Keyword Input_Imputed_Info_File not in batch file, Imputed Info file assumed to be unspecified.
## =====
## Analysis Class: Dump.
## Quantitative   Input Missing Value   -9
## Affection      Input Missing Value    "-9"
## Quantitative   Output Missing Value    "*"
## Affection      Output Missing Value    "*"
## Input Format: PLINK PED format (ped)
## Pedigree and map files specified as PLINK format.
## omit, penetrance, and frequency files are always in Mega2 format.
## Input files will be read in as PLINK or Mega2 format files as appropriate.
## Reading PLINK map file for names: Mega2r.map
## Reading map file Mega2r.map ... (4 columns)
## Input Map name: Map, type: average genetic map, units: kosambi centiMorgans
## Input Map name: BP, type: physical map
## Found 2 possible maps in the Mega2r.map file.
## Now checking each record in map file Mega2r.map ...
## Done reading map file: Mega2r.map
##
## =====
```

```

## Total number of loci = 1001
## 1 trait locus
##      1 Affection status locus:
##      default
##      1000 Marker loci
## Number of loci found per chromosome (chromosome:number)
##      1:1000
## =====
## WARNING: No frequency file provided.
## WARNING: Allele frequencies for these will be estimated from data.
## Trait 'default' will be assigned the default penetrance: (0.0500 0.9000 0.9000)
## Reading PLINK .ped file: Mega2r.ped (2006 columns).
## 1000 (of 1000) markers to be included from Mega2r.map
## Reading pedigree information from Mega2r.ped
## 1380 individuals read from Mega2r.ped
## 1380 individuals with nonmissing phenotypes
## 105 cases, 1275 controls, 0 missing
## 620 males, 760 females, 0 of unspecified sex
## 0 founders, 1380 non-founders found
## =====
## Input pedigree data contains:
## Input pedigree file is in PLINK-fam format.
##
##                                Marker Genotypes
##                                Fully   Half
## Pedigrees   People   Males   Females   Typed   Typed   Total
## TOTAL       20       1380    620     760     1380000    0     1380000
## Typed       20       1380    620     760
## Untyped     0        0       0       0
## =====
## Pedigree exclusion option : Include all pedigrees whether typed or not.
## Count option: all alleles
## Count half-typed individuals' alleles : no
## =====
## Recoding pedigree genotypes ...
## =====
## Pedigree data summary after recoding:
## Input pedigree file is in PLINK-fam format.
##
##                                Marker Genotypes
##                                Fully   Half
## Pedigrees   People   Males   Females   Typed   Typed   Total
## TOTAL       20       1380    620     760     1380000    0     1380000
## Typed       20       1380    620     760
## Untyped     0        0       0       0
## =====
## Created linkage ped tree
## Done checking locus integrity.
## Checking pedigree integrity...
## Done checking pedigree integrity.
## =====
## =====
## Pedigree statistics after selecting chromosomes and marker loci:
## Input pedigree file is in post-makeped format.
##
##                                Marker Genotypes
##                                Fully   Half

```

```

##      Pedigrees   People   Males   Females           Typed   Typed   Total
## TOTAL         20     1380     620     760       1380000         0  1380000
## Typed         20     1380     620     760
## Untyped        0         0         0         0
## =====
## Database file "seqsimr.db" will be backed up.
## Moved existing seqsimr.db to seqsimr.db.old
## Dumping SQLite3 DB to file "seqsimr.db"
## =====
## See run summaries in directory 2017-6-16-14-55
##      MEGA2.LOG, MEGA2.RECODE, MEGA2.ERR, MEGA2.KEYS
## The script 'mega2log2html.pl' exited normally.
## To view the HTML-formatted run summaries, open
## /Users/rbaron/mega2/bb/srcdir/R/mega2tutorial/2017-6-16-14-55/MEGA2run.html
## in a web browser.
## =====

```

If you do not provide the argument, Mega2 will proceed to ask a series of questions to collect the needed information to produce a database. In addition, it will create a Mega2.BATCH file, we've suggested you use. You can look at the "Quick Start" documentation https://watson.hgen.pitt.edu/docs/mega2_html/mega2.html Section 4. to better understand the interactive process.

This MEGA2.BATCH.seqsimr file begins with a rather long comment indicating the keyword values that may be set and their default value. Toward the end of the file, we set the inputs as **Mega2r.ped** and **Mega2r.map**, indicate the input is PLINK ped format with parameters, and indicate that Mega2 should produce a database called **seqsimr.db**, etc. (These particular items are in bold face text below.)

```

Input_Database_Mode=1
Input_Format_Type=4
Input_Pedigree_File=Mega2r.ped
Input_PLINK_Map_File=Mega2r.map
Output_Path=.
Input_Path=.
PLINK_Args= -cM -missing-phenotype -9 -trait default
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Value_Marker_Compression=1
Analysis_Option=Dump
Value_Missing_Quant_On_Input=-9.000000
Value_Missing_Affect_On_Input=-9
Count_Genotypes=4
Count_Halftyped=no
Value_Genetic_Distance_Index=0
Value_Genetic_Distance_SexTypeMap=0
Value_Base_Pair_Position_Index=1
Default_Reset_Invalid=no
DBfile_name=seqsimr.db
Default_Outfile_Names=yes

```

You will eventually have to run Mega2 to convert your data. But if you want to temporarily avoid the above processing, notice that in the mega2tutorial directory, there is a seqsimr.db file.

5.2 Reading a Mega2 database into R

After you have the database, start up the R program. The first R commands you should type are:

```
library(mega2r)
ENV = read.Mega2DB("seqsimr.db", verbose = TRUE)
```

The first argument should be the path to the database. Providing the optional argument, `verbose`, causes the `read` function to summarize the tables created, their fields and their sizes. Finally, an “R environment” that contains the database tables is returned. (If you are unfamiliar with environments, you can think of them as data frames. In fact, `ENV$locus_table` will access the `locus_table` variable from `ENV` as though fetching an “observation” from a data frame. The difference is when you change a data frame passed to a function, the change does not affect the original data frame. Only the function’s local value is changed; ALL changes are forgotten when the function exits. If you change the data in an environment passed to a function, the change is permanent.)

```
library(mega2r)
ENV = read.Mega2DB("seqsimr.db", verbose = TRUE)

## int_table      34  3
## int_table      pId key value
##
## pedigree_table  20  8
## pedigree_table  pId Num EntryCnt      Name      PedPre  OriginalID  origped pedigree_link
##
## person_table 1380   11
## person_table  pId UniqueID      OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link  person_
##
## pedigree_brkloop_table  20  8
## pedigree_brkloop_table  pId Num EntryCnt      Name      PedPre  OriginalID  origped pedigree_link
##
## person_brkloop_table 1380   11
## person_brkloop_table  pId UniqueID      OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link
##
## locus_table  1001   5
## locus_table  pId LocusName  Type      AlleleCnt  locus_link
##
## allele_table 2002   5
## allele_table pId AlleleName  Frequency  indexX  locus_link
##
## marker_table 1000   7
## marker_table pId MarkerName  pos_avg pos_female pos_male  chromosome  locus_link
##
## markerscheme_table  1000  4
## markerscheme_table  pId key allele1 allele2
##
## map_table  2000   6
## map_table  pId marker  map position  pos_female pos_male
##
## mapnames_table  2  6
## mapnames_table  pId map sex_averaged_map  male_sex_map  female_sex_map  name
##
## traitaff_table  1  4
```



```
## traitaff_table  pId ClassCnt    PenCnt  locus_link
##
## affectclass_table    1    9
## affectclass_table    pId MaleDef FemaleDef    AutoDef MalePen FemalePen    AutoPen locus_link  class_l
##
## phenotype_table  1380    4
## phenotype_table  pId person_link bytes    data
##
## genotype_table    1380    5
## genotype_table    pId person_link chr bytes    data
```

XXX We need to make two observations that apply to all Mega2R functions: The first is that when `verbose` is set in the initial `read.Mega2DB`, the value will be remembered. It may be used by any subsequent function. If `verbose` is `TRUE`, Mega2R functions will possibly print out far more diagnostic information than you might need, especially on subsequent runs. The second is that all Mega2R functions that do not return an environment, need to have an environment supplied as an argument. There are two ways to accomplish this. If you read “seqsimr.db” into the variable `seqsimr`, then you could supply as the named argument **envir** the value `seqsimr` in the Mega2R function as in:

```
showMega2ENV(envir = seqsimr)
```

The second choice is a bit of a “hack” but it is very convenient. Every Mega2R function (that does not return an environment) has a named argument defined as

```
envir = ENV
```

as in

```
showMega2ENV = function(envir = ENV) { ... }
```

The code above, sets global variable **ENV** to the local `envir` variable. And the argument evaluation mechanism of R will supports this usage; even though **ENV** is not known until runtime.

5.2.1 Back to more examples.

The `ls` function will show you all the variables in an environment. (You probably used it without arguments to show you the variables in the `.GlobalEnv`.) Type:

```
ls(ENV)
```

```
## [1] "LocusCnt"           "MARKER_SCHEME"
## [3] "PhenoCnt"           "affectclass_table"
## [5] "allele_table"       "entrezGene"
## [7] "fam"                "int_table"
## [9] "locus_allele_table" "locus_table"
## [11] "map_table"          "mapnames_table"
## [13] "marker_table"       "markers"
## [15] "markerscheme_table" "pedigree_brkloop_table"
## [17] "pedigree_table"     "person_brkloop_table"
## [19] "person_table"       "phenotype_table"
## [21] "refIndices"         "refRanges"
## [23] "traitaff_table"     "txdb"
## [25] "unified_genotype_table" "verbose"
```

5.2.2 A more informative overview of the data base can be seen by:

```
showMega2ENV()
```

```
## locus count: 1001; phenotype count: 1; compression: 2 bits
## marker count: 1000; sample count: 1380
##
## genetic and physical maps:
## map name map number
## 1 Map 0
## 2 BP 1
##
## Phenotypes:
## Index Name Type
## 1 1 default affection
##
##
## basic tables:
## rows cols
## affectclass_table 1 9
## allele_table 2002 5
## int_table 34 3
## locus_table 1001 5
## map_table 2000 6
## mapnames_table 2 6
## marker_table 1000 8
## markerscheme_table 1000 4
## pedigree_brkloop_table 20 8
## pedigree_table 20 8
## person_brkloop_table 1380 11
## person_table 1380 11
## phenotype_table 1380 4
## traitaff_table 1 4
##
## derived tables:
## rows cols
## fam 1380 8
## markers 1000 5
## unified_genotype_table 1380 2
```

5.2.3 Use standard R commands to examine the created data frames

```
str(ENV$locus_table)
```

```
## 'data.frame': 1001 obs. of 5 variables:
## $ pId : int 1 2 3 4 5 6 7 8 9 10 ...
## $ LocusName : chr "default" "snp1" "snp2" "snp3" ...
## $ Type : int 2 4 4 4 4 4 4 4 4 4 ...
## $ AlleleCnt : int 2 2 2 2 2 2 2 2 2 2 ...
## $ locus_link: int 0 1 2 3 4 5 6 7 8 9 ...
```

5.3 Running pedgene using the mega2pedgene R package

mega2pedgene is the exception that proves the rule. Rather than use the read.Mega2DB function described above and as used by all the other packages, we use `init_pedgene`. This function calls a utility function created for read.Mega2DB. Then it creates, edits, and rewrites the **fam** data frame to purge persons with unknown case/control status. (**fam** merges data from the pedigree_table, person_table and phenotype_table as you might expect.) Finally, it calculates and stores some values that will be used repeatedly. viz:

```
library(mega2pedgene)
ENV = init_pedgene("seqsimr.db", verbose = T)

## int_table      34  3
## int_table      pId key value
##
## pedigree_table  20  8
## pedigree_table  pId Num EntryCnt      Name      PedPre  OriginalID  origped pedigree_link
##
## person_table 1380   11
## person_table  pId UniqueID      OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link  person_
##
## pedigree_brkloop_table  20  8
## pedigree_brkloop_table  pId Num EntryCnt      Name      PedPre  OriginalID  origped pedigree_link
##
## person_brkloop_table 1380   11
## person_brkloop_table  pId UniqueID      OrigID  FamName PerPre  ID  Father  Mother  Sex pedigree_link
##
## locus_table  1001   5
## locus_table  pId LocusName  Type      AlleleCnt  locus_link
##
## allele_table 2002   5
## allele_table pId AlleleName  Frequency  indexX  locus_link
##
## marker_table 1000   7
## marker_table pId MarkerName  pos_avg pos_female pos_male  chromosome  locus_link
##
## markerscheme_table  1000   4
## markerscheme_table  pId key allele1 allele2
##
## map_table  2000   6
## map_table  pId marker  map position  pos_female pos_male
##
## mapnames_table  2  6
## mapnames_table  pId map sex_averaged_map  male_sex_map  female_sex_map  name
##
## traitaff_table  1  4
## traitaff_table  pId ClassCnt  PenCnt  locus_link
##
## affectclass_table  1  9
## affectclass_table  pId MaleDef FemaleDef  AutoDef MalePen FemalePen  AutoPen locus_link  class_1
##
## phenotype_table 1380   4
## phenotype_table  pId person_link bytes  data
##
## genotype_table  1380   5
```

```
## genotype_table  pId person_link chr bytes  data
```

Mega2R has an internal default list of the chromosome and base pair ranges for a number of gene transcripts. These transcripts come from the UCSC Genome Browser reference assembly GRCH37. The list was further modified to eliminate multiple records of the same gene with the exact same transcript start and transcript end. This constitutes about 30,000 records. The function `run_pedgene` examines the first 100 transcripts and prints the results. However, our test data is from part of the first chromosome only; the first 100 transcripts find no markers to run pedgene on.

```
run_pedgene()
```

When we perform more thorough runs, below, you will see many reports of “No markers in range”, but occasionally you will see a listing of a gene name, markers, and count of 1/1, 1/2 and 2/2 genotypes, viz.

```
CEP104 snp24027 1280 97 3
CEP104 snp24788 1375 5 0
```

The genotypes for these markers, along with the markers info are fed to the call back function `DOPedgene`. `DOPedgene` converts the genotype nucleotides to the count of the minor allele and then runs pedgene for several different marker weights. The results are automatically stored in a file, “k_Schaid_rare.txt”. Lines in that file have a prefix of chromosome, gene, number of markers and base pair range followed by 6 p-values. Two for each of three weightings of the markers. It is printed as well.

```
chr1 CEP104 2 3728644 3773797 0.8939136 0.8047689 0.8155284 0.6827252 0.9473635 0.9365409 1
```

It would be better to run pedgene on all the default transcript entries. So type:

```
# we will skip this line for the Rmd document production because it takes too
# long
applyFnToRanges(DOPedgene, ENV$refRanges, ENV$refIndices, envir = ENV)
```

If you run the above test, you will see that gene `ROR1` has some p-values less than 0.01, `FAM129A` less than 0.02, and `NEK7` and `IGSF21` less than 0.03. If we knew this fact to start with, and hence these were the only genes we were interested in.

NOTE: You will observe that for several genes there are a few transcripts with different start and end values. A shorter test would be

```
# turn off verbose
ENV$verbose = FALSE
applyFnToRanges(DOPedgene, ENV$refRanges[1:300, ], ENV$refIndices, envir = ENV)
```

Alternatively, you may try searching by Gene. Here the default transcript database is `TxDb.Hsapiens.UCSC.hg19.knownGene` from Bioconductor. Of course you can change it.

```
applyFnToGenes(DOPedgene, genes_arg = c("ROR1", "FAM129A", "NEK7", "IGSF21"), envir = ENV)
```

```
## 'select()' returned 1:many mapping between keys and columns
## 'select()' returned 1:many mapping between keys and columns
```

or

```
applyFnToGenes(D0pedgene, genes_arg = c("GDAP2"), envir = ENV)
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

```
## 'select()' returned 1:many mapping between keys and columns
```

5.4 Writing a VCF File using the mega2vcf R package

First create the directory “vcfr”; so we can keep all the generated data in one place and then load the library.

Mega2VCF takes an argument which is the prefix used on all the files it generates. We have chosen to make that a path that includes a directory. Mega2VCF is explained in greater depth in the Implementation Section. The Mega2VCF function assumes the environment is contained in the global variable ENV. If this is not true supply an additional argument “envir” set to the environment you wish to use.

Note, that Mega2VCF will create the expected .vcf file as well as a .fam file (we do linkage), a .phe file (phenotypes) and a few others. These files are listed below.

Creating the VCF file and related files is accomplished by typing:

```
if (!dir.exists("vcfr")) dir.create("vcfr")
library(mega2vcf)
Mega2VCF("vcfr/vcf.01", ENV$markers[ENV$markers$chromosome == 1, ])
list.files("vcfr")
```

```
## [1] "vcf.01.fam" "vcf.01.freq" "vcf.01.map" "vcf.01.pen" "vcf.01.phe"
## [6] "vcf.01.vcf"
```

Read the package code to see how these files are created from the data frames.

5.5 Creating a GenABEL gwaa.data-class object using the mega2genabel R package

GenABEL can process PLINK .tped/.tfam files with the function *convert.snp.tped* to create a gwaa.data-class object. Currently, we use this mechanism to convert our data frames to a gwaa.data-class object. The argument to Mega2GenABEL is a file name prefix that will be used to name the .tped/.fam files that are created as an intermediate to be read by GenABEL <http://www.genabel.org/> <https://cran.r-project.org/web/packages/GenABEL/index.html> to produce a GenABEL “raw” file. Then the “raw” file and a generated phenotype file are processed by *load.gwaa.data* to yield a gwaa.data-class object. (See the Implementation Section.)

This task is analogous to the previous exercise. Type:

```
library(mega2genabel)
seqsingwaa = Mega2GenABEL("TMP")
```

```
## Reading individual ids from file 'TMP.tfam' ...
## ... done. Read 1380 individual ids from file 'TMP.tfam'
## Reading genotypes from file 'TMP.tped' ...
## ...done. Read 1000 SNPs from file 'TMP.tped'
## Writing to file 'TMPtped.raw' ...
```

```
## ... done.
## ids loaded...
## marker names loaded...
## chromosome data loaded...
## map data loaded...
## allele coding data loaded...
## strand data loaded...
## genotype data loaded...
## snp.data object created...
## assignment of gwaa.data object FORCED; X-errors were not checked!
```

```
list.files(pattern = "TMP.*")
```

```
## [1] "TMP.phe"      "TMP.tfam"      "TMP.tped"      "TMPtped.raw"
```

```
str(seqsingwaa)
```

```
## Formal class 'gwaa.data' [package "GenABEL"] with 2 slots
## ..@ phdata:'data.frame': 1380 obs. of 3 variables:
## .. ..$ id : chr [1:1380] "1SAP039_H05-0107" "1SAP039_H05-0106" "1SAP039_SAP039F14" "1SAP039_SAP039F14" ...
## .. ..$ sex : int [1:1380] 0 1 1 1 1 0 0 0 0 0 ...
## .. ..$ default: int [1:1380] 1 1 1 1 1 1 1 1 1 1 ...
## ..@ gtdata:Formal class 'snp.data' [package "GenABEL"] with 11 slots
## .. ..@ nbytes : num 345
## .. ..@ nids : int 1380
## .. ..@ nsnp : int 1000
## .. ..@ idnames : chr [1:1380] "1SAP039_H05-0107" "1SAP039_H05-0106" "1SAP039_SAP039F14" "1SAP039_SAP039F14" ...
## .. ..@ snpnames : chr [1:1000] "snp1" "snp2" "snp3" "snp4" ...
## .. ..@ chromosome: Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## .. ..@ map : Named num [1:1000] 2 3 4 5 5 6 8 11 12 17 ...
## .. ..@ coding : Formal class 'snp.coding' [package "GenABEL"] with 1 slot
## .. ..@ strand : Formal class 'snp.strand' [package "GenABEL"] with 1 slot
## .. ..@ male : Named int [1:1380] 0 1 1 1 1 0 0 0 0 0 ...
## .. ..@ gtps : Formal class 'snp.mx' [package "GenABEL"] with 1 slot
## .. ..@ .Data: raw [1:345, 1:1000] 59 55 55 55 ...
```

You can use any of the GenABEL provided functions on your data, seqsingwaa.

Note: There are several data formats that Mega2 can read and transform into a database that are not directly acceptable to GenABEL. Any M1ega2 database that can be read into R, can be transformed to GenABEL following this example with seqsimr.db.

6 Regression

To verify that the R packages described above have worked correctly, we can compare their output files to those created by Mega2 itself, to verify that they are, as expected, identical. This type of testing is known as regression testing.

6.1 VCF regression

It is time to use the MEGA2.BATCH.vcf file from the mega2rtutorial. This will use the C++ Mega2 program to populate the vcf directory with the same set of files that the R program created in the vcfr directory. At the command prompt, type:

```
mkdir vcf
mega2 MEGA2.BATCH.vcf
```

The abbreviated MEGA2.BATCH.vcf file is below (notice that there are no INPUT FILES, just a database file):

```
Input_Database_Mode=2
Align_Strand_Input=no
Output_Path=vcf
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Analysis_Option=VCF
Value_Missing_Quant_On_Output=-9
Value_Missing_Affect_On_Output=-9
DBfile_name=seqsimr.db
Chromosome_Single=1
Traits_Combine=1 2 e
file_name_stem=vcf
human_genome_build=B37
VCF_output_file_type=1
VCF_Allele_Order=Original_Order
Default_Outfile_Names=yes
```

```
mkdir vcf
mega2 MEGA2.BATCH.vcf
```

```
## mkdir: vcf: File exists
## =====
##                               MEGA2 4.9.2
##   Copyright (C) 1999-2017 Robert Baron, Justin R. Stickel, Charles P. Kollar,
##   Nandita Mukhopadhyay, Lee Almasy, Mark Schroeder, William P. Mulvihill,
##   Daniel E. Weeks, and University of Pittsburgh
##
##   Last updated: Jun 13 2017, 09:36:42 , valid until June 15, 2018.
##   Compiled with gcc version 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)
##
##   Mega2 comes with ABSOLUTELY NO WARRANTY.
##   See LICENSE.txt for terms of copying, modifying & redistributing Mega2.
## =====
## NOTE: For humans, chromosome 23 codes for X, 24 codes for Y and 25 codes for XY.
##
## Run date:                2017-6-16-14-55
##
## Running Mega2 in batch mode from MEGA2.BATCH.vcf.
## Analysis option read in from batch file.
## Chromosome(s) and markers read in from batch file.
```

```

## Trait selection(s) read in from batch file.
## =====
## Analysis Class: VCF.
## Quantitative Output Missing Value      "-9"
## Affection      Output Missing Value      "-9"
## Reading SQLite3 DB from file "seqsimr.db"
## =====
## The path to this SQLite3 database is seqsimr.db.
## This database was created using Mega2 version 4.9.2.
## This database was created using      SQLite3 3.9.2 on 2017-6-16-14-55.
## This database was processed using SQLite3 3.9.2 on 2017-6-16-14-55.
## This database was created from PLINK PED format data using the following files:
##      Pedigree file      Mega2r.ped
##      PLINK Map file      Mega2r.map
## This database contains:
## 1380 persons (20 pedigrees)
## 1000 markers
## 1 trait
## genetic distance(map name/type) "Map"/kosambi, Sex map type AVERAGED_MAP
## base pair distance(map name) "BP"
##
## =====
## 1 trait locus
##      1 Affection status locus:
##      default
## =====
## Selected map Map.
## Selected chromosome 1
## Output will combine markers and the following selected traits:
##      default [MARKERS]
## After selecting traits and covariates
## 1 trait locus
##      1 Affection status locus:
##      default
## =====
## Pedigree statistics after selecting chromosomes and marker loci:
## Input pedigree file is in post-makeped format.
##
##                               Marker Genotypes
##                               Fully      Half
##      Pedigrees  People  Males  Females  Typed  Typed  Total
## TOTAL          20     1380    620    760    1380000      0  1380000
## Typed          20     1380    620    760
## Untyped         0         0      0      0
## =====
## Mega2 created the following file(s) for VCF Format:
## VCF file created using allele ordering setting: Original_Order
##      VCF format file:      vcf/vcf.01.vcf
##      VCF pedigree file:     vcf/vcf.01.fam
##      VCF phenotype file:    vcf/vcf.01.phe
##      VCF map file:          vcf/vcf.01.map
##      VCF freq file:         vcf/vcf.01.freq
##      VCF pen file:          vcf/vcf.01.pen
##
## SQLite3 database "seqsimr.db" was processed to generate this output.

```



```
## Output is in vcf
## =====
## See run summaries in directory 2017-6-16-14-55
##   MEGA2.LOG, MEGA2.RECODE, MEGA2.ERR, MEGA2.KEYS
## The script 'mega2log2html.pl' exited normally.
## To view the HTML-formatted run summaries, open
## /Users/rbaron/mega2/bb/srcdir/R/mega2rtutorial/2017-6-16-14-55/MEGA2run.html
## in a web browser.
## =====
## If you use Mega2 as part of a published work, please reference
## Baron RV, Kollar C, Mukhopadhyay N, Weeks DE
## Mega2: validated data-reformatting for linkage and association analyses
## Source Code for Biology and Medicine.2014, 9:26
## DOI: 10.1186/s13029-014-0026-y
## as well as the version used, which is currently Version 4.9.2
## =====
```

You should compare the two directories: vcf and vcfr. Please add the -w flag to the diff command. This causes the comparison to ignore differences in white space. You may notice that expressions that contain dates that are different between the two directories. Type:

```
diff -wrs vcf vcfr
```

```
## Files vcf/vcf.01.fam and vcfr/vcf.01.fam are identical
## Files vcf/vcf.01.freq and vcfr/vcf.01.freq are identical
## Files vcf/vcf.01.map and vcfr/vcf.01.map are identical
## Files vcf/vcf.01.pen and vcfr/vcf.01.pen are identical
## Files vcf/vcf.01.phe and vcfr/vcf.01.phe are identical
## Files vcf/vcf.01.vcf and vcfr/vcf.01.vcf are identical
```

6.2 GenABEL regression

This test is a bit more complicated. First, we load GenABEL and access its data:

```
library(GenABEL)
data(srdata)
```

We also load the mega2genabel library and dump the GenABEL data as PLINK .ped/.map/.phe files

```
library(mega2genabel)
dmpPed()
```

Then we switch to the command line and run:

```
mega2 MEGA2.BATCH.srdata
```

6.2.1 (abbreviated contents of MEGA2.BATCH.srdata file)

```
Input_Database_Mode=1
Input_Format_Type=4
Input_Pedigree_File=srdata.ped
```

```

Input_PLINK_Map_File=srdta.map
Input_Phenotype_File=srdta.phe
Output_Path=.
Input_Path=.
PLINK_Args= -missing-phenotype -9 -trait default
Input_Untyped_Ped_Option=2
Input_Do_Error_Sim=no
AlleleFreq_SquaredDev=999999999.000000
Value_Marker_Compression=1
Analysis_Option=Dump
Value_Missing_Quant_On_Input=-9.000000
Value_Missing_Affect_On_Input=-9
Count_Genotypes=4
Count_Halftyped=no
Value_Genetic_Distance_Index=0
Value_Genetic_Distance_SexTypeMap=0
Value_Base_Pair_Position_Index=1
Default_Reset_Invalid=no
DBfile_name=srdta.db
Default_Outfile_Names=yes

```

to produce the Mega2 database srdta.db.

mega2 MEGA2.BATCH.srdta

```

## =====
##                               MEGA2 4.9.2
##   Copyright (C) 1999-2017 Robert Baron, Justin R. Stickel, Charles P. Kollar,
##   Nandita Mukhopadhyay, Lee Almasy, Mark Schroeder, William P. Mulvihill,
##   Daniel E. Weeks, and University of Pittsburgh
##
##   Last updated: Jun 13 2017, 09:36:42 , valid until June 15, 2018.
##   Compiled with gcc version 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)
##
##   Mega2 comes with ABSOLUTELY NO WARRANTY.
##   See LICENSE.txt for terms of copying, modifying & redistributing Mega2.
## =====
## NOTE: For humans, chromosome 23 codes for X, 24 codes for Y and 25 codes for XY.
##
## Run date:                2017-6-16-14-56
##
## Running Mega2 in batch mode from MEGA2.BATCH.srdta.
## Input filenames and missing value indicator read in from batch file.
## Dump Analysis option read in from batch file.
## WARNING: Locus selections not specified in batch file.
## WARNING: Going to Reorder menu.
## WARNING: Trait selections not specified in batch file.
## WARNING: Going to Trait selection menu.
## =====
## Keyword Input_Locus_File not in batch file, Locus file assumed to be unspecified.
## Keyword Input_Map_File not in batch file, Map file assumed to be unspecified.
## Keyword Input_Omit_File not in batch file, Omit file assumed to be unspecified.
## Keyword Input_Frequency_File not in batch file, Frequency file assumed to be unspecified.

```

```

## Keyword Input_Penetrance_File not in batch file, Penetrance file assumed to be unspecified.
## Keyword Input_Aux_File not in batch file, Aux file assumed to be unspecified.
## Keyword Input_Imputed_Info_File not in batch file, Imputed Info file assumed to be unspecified.
## =====
## Analysis Class: Dump.
## Quantitative Input Missing Value -9
## Affection Input Missing Value "-9"
## Quantitative Output Missing Value "*"
## Affection Output Missing Value "*"
## Input Format: PLINK PED format (ped)
## Pedigree and map files specified as PLINK format.
## omit, penetrance, and frequency files are always in Mega2 format.
## Input files will be read in as PLINK or Mega2 format files as appropriate.
## reading phenotype file srdta.phe ... (5 columns)
## Reading PLINK map file for names: srdta.map
## Reading map file srdta.map ... (4 columns)
## Input Map name: Map, type: average genetic map, units: kosambi Morgans
## Input Map name: BP, type: physical map
## Found 2 possible maps in the srdta.map file.
## Now checking each record in map file srdta.map ...
## Done reading map file: srdta.map
##
## =====
## Total number of loci = 839
## 6 trait loci
## 2 Affection status loci:
## bt default
## 4 Quantitative loci:
## age qt1 qt2 qt3
## 833 Marker loci
## Number of loci found per chromosome (chromosome:number)
## 1:833
## =====
## WARNING: No frequency file provided.
## WARNING: Allele frequencies for these will be estimated from data.
## Trait 'bt' will be assigned the default penetrance: (0.0500 0.9000 0.9000)
## Trait 'default' will be assigned the default penetrance: (0.0500 0.9000 0.9000)
## Reading PLINK .ped file: srdta.ped (1672 columns).
## 833 (of 833) markers to be included from srdta.map
## Reading pedigree information from srdta.ped
## 2500 individuals read from srdta.ped
## 2500 individuals with nonmissing phenotypes
## 0 cases, 1244 controls, 1256 missing
## 1275 males, 1225 females, 0 of unspecified sex
## 0 founders, 2500 non-founders found
## =====
## Input pedigree data contains:
## Input pedigree file is in PLINK-fam format.
##
## Marker Genotypes
## Fully Half
## Typed Typed Total
## TOTAL 2500 2500 1275 1225 1978958 0 2082500
## Typed 2500 2500 1275 1225
## Untyped 0 0 0 0

```

```

## =====
## Pedigree exclusion option : Include all pedigrees whether typed or not.
## Count option: all alleles
## Count half-typed individuals' alleles : no
## =====
## Recoding pedigree genotypes ...
## =====
## Pedigree data summary after recoding:
## Input pedigree file is in PLINK-fam format.
##
##                               Marker Genotypes
##                               Fully   Half
## Pedigrees  People  Males  Females  Typed  Typed  Total
## TOTAL      2500    2500    1275    1225    1978958  0    2082500
## Typed      2500    2500    1275    1225
## Untyped     0       0       0       0
## =====
## Created linkage ped tree
##
## ===== Messages of type "quant_stat":
## -----
## Per-pedigree quantitative phenotype summary:
## Pedigree      Mean      Std Dev      Minimum      Maximum      #Phenotypes
## -----
## age
## -----
## -----
## p1           43.40000    0.00000    43.40000    43.40000      1
## p2           48.20000    0.00000    48.20000    48.20000      1
## p3           37.90000    0.00000    37.90000    37.90000      1
## p4           53.80000    0.00000    53.80000    53.80000      1
## p5           47.50000    0.00000    47.50000    47.50000      1
## p6           45.00000    0.00000    45.00000    45.00000      1
## p7           52.00000    0.00000    52.00000    52.00000      1
## p8           42.50000    0.00000    42.50000    42.50000      1
## p9           29.70000    0.00000    29.70000    29.70000      1
## ===== Too many "quant_stat" records, display is temporarily suspended ..
## qt1
## -----
## -----
## p1           -0.58000    0.00000    -0.58000    -0.58000      1
## p2            0.80000    0.00000     0.80000     0.80000      1
## p3           -0.52000    0.00000    -0.52000    -0.52000      1
## p4           -1.55000    0.00000    -1.55000    -1.55000      1
## p5            0.25000    0.00000     0.25000     0.25000      1
## p6            0.15000    0.00000     0.15000     0.15000      1
## p7           -0.56000    0.00000    -0.56000    -0.56000      1
## p8            0.00000    0.00000     0.00000     0.00000      0
## p9           -2.26000    0.00000    -2.26000    -2.26000      1
## ===== Too many "quant_stat" records, display is temporarily suspended ..
## qt2
## -----
## -----
## p1            4.46000    0.00000     4.46000     4.46000      1
## p2            6.32000    0.00000     6.32000     6.32000      1

```

```

## p3          3.26000  0.00000  3.26000  3.26000  1
## p4          888.00000 0.00000 888.00000 888.00000 1
## p5          5.70000  0.00000  5.70000  5.70000  1
## p6          4.65000  0.00000  4.65000  4.65000  1
## p7          4.64000  0.00000  4.64000  4.64000  1
## p8          5.77000  0.00000  5.77000  5.77000  1
## p9          0.71000  0.00000  0.71000  0.71000  1
## ===== Too many "quant_stat" records, display is temporarily suspended ..
## qt3
## -----
## -----
## p1          1.43000  0.00000  1.43000  1.43000  1
## p2          3.90000  0.00000  3.90000  3.90000  1
## p3          5.05000  0.00000  5.05000  5.05000  1
## p4          3.76000  0.00000  3.76000  3.76000  1
## p5          2.89000  0.00000  2.89000  2.89000  1
## p6          1.87000  0.00000  1.87000  1.87000  1
## p7          2.49000  0.00000  2.49000  2.49000  1
## p8          2.68000  0.00000  2.68000  2.68000  1
## p9          1.45000  0.00000  1.45000  1.45000  1
## ===== Too many "quant_stat" records, display is temporarily suspended ..
## ===== 2501 total records of type "quant_stat" are in MEGA2.LOG
##
##
## ===== Messages of type "quant_stat_sum":
##               Quantitative trait phenotype statistics
## -----
## QTL          Missing  Minimum  Maximum  Total      Pedigrees   Total
##               pedigrees phenotyped phenotypes
## -----
## age          0    24.100    71.600    2500        2500        2500
## qt1          3    -4.600     3.200    2500        2497        2497
## qt2          0     0.000    888.000    2500        2500        2500
## qt3         11    -1.970     6.340    2500        2489        2489
## NOTE: The Missing QTL value on input has been assigned as '-9.00'.
## -----
## QTL          Mean    Std Dev  Skewness  Kurtosis
## -----
## age          50.038    7.060    0.003    -0.063
## qt1          -0.298    1.001   -0.075    0.126
## qt2          6.122    30.601   28.734   825.077
## qt3          2.609    1.101    0.033   -0.090
## =====
## ===== 4 total records of type "quant_stat_sum" are in MEGA2.LOG
##
## Done checking locus integrity.
## Checking pedigree integrity...
## Done checking pedigree integrity.
## =====
## =====
## Pedigree statistics after selecting chromosomes and marker loci:
## Input pedigree file is in post-makeped format.
##
##                               Marker Genotypes
##                               Fully    Half

```

```
##      Pedigrees   People   Males   Females      Typed      Typed      Total
## TOTAL      2500      2500      1275      1225      1978958         0      2082500
## Typed      2500      2500      1275      1225
## Untyped        0         0         0         0
## =====
## Database file "srdta.db" will be backed up.
## Moved existing srdta.db to srdta.db.old
## Dumping SQLite3 DB to file "srdta.db"
## =====
## See run summaries in directory 2017-6-16-14-56
##      MEGA2.LOG, MEGA2.RECODE, MEGA2.ERR, MEGA2.KEYS
## The script 'mega2log2html.pl' exited normally.
## To view the HTML-formatted run summaries, open
## /Users/rbaron/mega2/bb/srcdir/R/mega2rtutorial/2017-6-16-14-56/MEGA2run.html
## in a web browser.
## =====
```

Then we go back into R and type:

```
ENV = read.Mega2DB("srdta.db")
mega = Mega2GenABEL("TMP2")
```

```
## Reading individual ids from file 'TMP2.tfam' ...
## ... done. Read 2500 individual ids from file 'TMP2.tfam'
## Reading genotypes from file 'TMP2.tped' ...
## ...done. Read 833 SNPs from file 'TMP2.tped'
## Writing to file 'TMP2tped.raw' ...
## ... done.
## ids loaded...
## marker names loaded...
## chromosome data loaded...
## map data loaded...
## allele coding data loaded...
## strand data loaded...
## genotype data loaded...
## snp.data object created...
## assignment of gwaa.data object FORCED; X-errors were not checked!
```

mega and *srdta* should be the same *gwaa.data*-class object. You can compare them however you prefer. But, *Mega2GenABELtst* can be used to compare the phenotype and genotype data, viz. in R type:

```
Mega2GenABELtst()
```

```
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
```

```
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
## [1] TRUE
```

7 Next steps

If you are left with questions as to how you can use Mega2R for your own research, you can:

1. Read our paper “The Mega2R suite of R packages: tools for accessing and processing common genetic data formats in R”, which extends this document by providing implementation details.
2. Read the source for the packages if you understand R; the code is not particularly subtle.
3. Write to the Mega2 discussion group: <https://groups.google.com/forum/#!forum/mega2-users>